

锂离子电池快充策略对寿命衰减的影响建模与优化

摘要

快充策略在缩短锂离子电池补能时间的同时会加速容量衰减，如何刻画快充策略与寿命衰减的定量关系并兼顾二者优化，是新能源汽车与储能系统的关键问题。本文基于 MIT-Stanford 锂离子电池循环老化数据集（49 块电池、9350 条循环记录、9 种两阶段快充策略），围绕快充策略参数（ C_1 、 Q_1 、 C_2 ）对寿命衰减的影响，建立了数据清洗、统计推断、寿命预测到多目标优化的完整流程，始终坚持可复现、留出验证与不确定性量化。

针对问题一：完成最小干预的数据清洗（修复 3 处异常），以两阶段函数型曲线模型为主模型、样条混合与多项式混合为候选，按整块电池留一验证（LOBO）估计各策略 SOH 曲线，主模型 LOBO RMSE 为 0.004325。三种模型的策略排序一致，典型长寿命策略为 $3_6C-80PER_3_6C$ 与 $5C_67PER_4C$ ，短寿命为 $3_7C_31PER_5_9C$ 与 $80PER_3_6C$ ；经 Holm 校正后无确证显著的策略对，但 20 组未校正 bootstrap 区间不跨零。

针对问题二：引入低维应力坐标（总电流应力 J 、高 SOC 加权应力 H 及高 SOC 分位应力 J_{high} ）建立约束退化混合模型。策略间末段退化率存在全局差异（诊断性尾部比例 5×10^{-5} ），高 SOC 分位应力与退化方向一致且 bootstrap 系数 100% 为正；但受固定协议组、样本量与方差不等限制，该诊断非确证性检验，选择校正后尾部比例为 0.065，故不能宣称 C_1, Q_1, C_2 的独立因果效应，本文如实报告该边界。

针对问题三：因部署时固定提供 150 个观测循环，按 $L = 150$ 外层 LOBO 冻结线性外推模型 $P1_linear$ ，策略等权 RMSE 为 0.000617；轨迹岭模型 C_ridge 作为多长度综合稳健性敏感性保留。给出 9 块测试电池 151–200 循环预测与保守点区间，80% 寿命仅作情景外推。

针对问题四：采用观测策略离散 Pareto 与整块电池 bootstrap。若优先缩短时间并接受约 0.13% 退化，推荐 $5_3C_54PER_4C$ （ $C_1 = 5.3, Q_1 = 54\%, C_2 = 4.0$ ）：较长寿命基准缩短约 24% 时间、退化仅增约 0.13 个百分点； $5C_67PER_4C$ 因成对 bootstrap 区间跨零而保留为近似并列敏感性项而非共同推荐。连续参数响应面因 oracle 压力测试失败被淘汰。

跨问题一致结论：充电策略标签与 SOH 退化相关，早期 SOH 动态是短期预测的主要信息，但稀疏混杂设计使结论须限制在已观测策略内，所有寿命终点均为外推估计。

关键字：锂离子电池 快充策略 函数型曲线 混合效应模型 留一验证 Pareto 优化

一、问题重述

1.1 问题背景

锂离子电池的快速充电能够显著缩短补能时间、提高使用便利性，但过高的充电电流会导致电池内部极化增强、热效应加剧与副反应加速，从而加快容量衰减、缩短电池寿命^[4]。电池寿命通常以健康状态 SOH 表征，定义为当前可用容量与初始容量之比，本题统一以 80% SOH 作为寿命终止阈值。

影响电池寿命的两个主要指标为充电倍率（C-rate）与荷电状态 SOC。不同 SOC 区间内电池对充电电流的敏感程度不同：在低 SOC 区间采用较大电流可能有助于缩短充电时间，而在中高 SOC 区间继续采用高倍率充电则可能加剧老化。本题采用 MIT-Stanford 公开数据集（49 块 A123 18650 磷酸铁锂电池，额定容量 1.1 Ah），各电池放电策略一致，差异体现在两阶段快充策略：先以 C_1 倍率从 0% 充电至 Q_1 ，再以 C_2 倍率从 Q_1 充电至 80%，最后以 1C 恒流-恒压（CC-CV）充至满电。

1.2 问题要求

- (1) 整理数据，分析不同快充策略下的寿命分布，提取并对比典型长寿命与短寿命策略；
- (2) 分析充电策略参数（ C_1, Q_1, C_2 ）对寿命衰减的影响，检验差异显著性及参数影响程度；
- (3) 基于 9 块测试电池截至第 150 循环的数据建立寿命预测模型，预测第 151–200 循环 SOH 并外推寿命，评价精度及早期数据长度的影响；
- (4) 设计兼顾充电时间与寿命衰减的策略优化方法，给出推荐策略并与典型策略对比。

1.3 数据特征

本数据集有三个关键特征：（1）9350 条循环记录是 49 块电池的重复测量，观测非独立，显著性检验与交叉验证必须按整块电池划分；（2）仅有 9 种策略， C_1, Q_1, C_2 联合变化，参数坐标高度相关，且存在新旧结构/数据批次混杂；（3）数据只观测至第 200 循环，SOH 最低约 0.95，无任何电池达到 80% 终点，因此寿命终点只能外推^[1]。

二、问题分析

2.1 问题一的分析

问题一是探索性建模。用函数型/混合效应模型估计各策略 SOH 曲线，按整块电池留一验证选模，以 bootstrap 量化不确定性，识别典型长/短寿命策略。

2.2 问题二的分析

问题二是统计推断。核心难点是稀疏混杂设计下参数效应不可识别。思路是构造物理启发的低维应力坐标，用约束退化混合模型刻画退化，用置换检验与选择校正给出稳健的显著性结论。需特别注意：固定协议组、样本量与方差不等时，置换检验是诊断性而非确证性工具^[6, 4]。

2.3 问题三的分析

问题三是预测。从早期循环提取轨迹特征，比较六类模型，用嵌套留一验证选择并冻结主模型，预测测试电池未来轨迹并给出区间；寿命终点作为情景外推。因部署时固定提供 150 个观测循环，模型选择应以 $L = 150$ 的实际部署窗口为准^[7, 10]。

2.4 问题四的分析

问题四是优化。在 9 个观测策略上做离散 Pareto 分析，用整块电池 bootstrap 评估前沿稳定性，给出条件约束下的推荐；连续参数搜索因设计不足被谨慎淘汰^[2, 3]。

三、模型假设

- (1) 假设电池容量测量误差为随机噪声，单次异常尖峰不代表真实寿命改善，可作为数据质量异常修复；
- (2) 假设各电池放电策略一致，电池间差异来源于充电策略与个体制造差异；
- (3) 假设 SOH 在观测窗口内近似平滑，可用函数型/混合效应模型描述，且平均 SOH 随循环单调下降；
- (4) 假设充电时间主要由两阶段恒流决定，恒压尾段与结构/批次影响作为经验项；
- (5) 假设推断单位是整块电池而非循环记录，显著性检验与重采样均按电池/策略聚类；
- (6) 忽略温度、内阻的精细机理，仅将其作为观测协变量^[8]。

四、符号说明

表 1 符号说明

符号	含义	单位
C_1	第一阶段充电倍率（低 SOC 区， $0 \sim Q_1$ ）	—
Q_1, q	阶段切换 SOC； $q = Q_1/100$	%
C_2	第二阶段充电倍率（中高 SOC 区， $Q_1 \sim 80\%$ ）	—
J	总电流应力 $qC_1 + (0.8 - q)C_2$	—
H	高 SOC 加权应力 $\frac{1}{2}[C_1q^2 + C_2(0.8^2 - q^2)]$	—
J_{high,s_0}	SOC 高于 s_0 区间的倍率应力	—
T_0	理想恒流充电时间	min
L	早期观测数据长度	循环
b_i	第 i 块电池的基线容量	Ah
$\text{SOH}_{i,t}$	第 i 块电池第 t 循环的健康状态	—
r_i	第 i 块电池的退化率（对数尺度）	—
D_p	策略 p 的 200 循环退化量	—
T_p	策略 p 的平均充电时间	min

五、数据预处理

5.1 原始数据核对

原始数据含电池级摘要表（49 行）与循环级时序表（9350 行）。核对确认 40 块完整电池包含第 1–200 循环，9 块测试电池（ $\text{prediction_test} = 1$ ）仅含第 1–150 循环；共 9 种充电策略。

5.2 异常识别与可审计修复

对循环数据逐电池检测局部异常：容量采用 ± 5 循环邻域的稳健 z 分数与相对偏差判定，内阻以 $IR \leq 0$ 判定为物理无效。修复采用相邻有效循环线性插值，共 3 处（表 2），原始值与修复标记全部保留。

表 2 数据清洗修复记录

电池	循环	变量	原始值	修复值	依据
1	12	容量/Ah	1.539054	1.074304	稳健 z 与相对偏差双超限
2	12	内阻/ Ω	0	0.016869	内阻必须为正
3	12	内阻/ Ω	0	0.016683	内阻必须为正

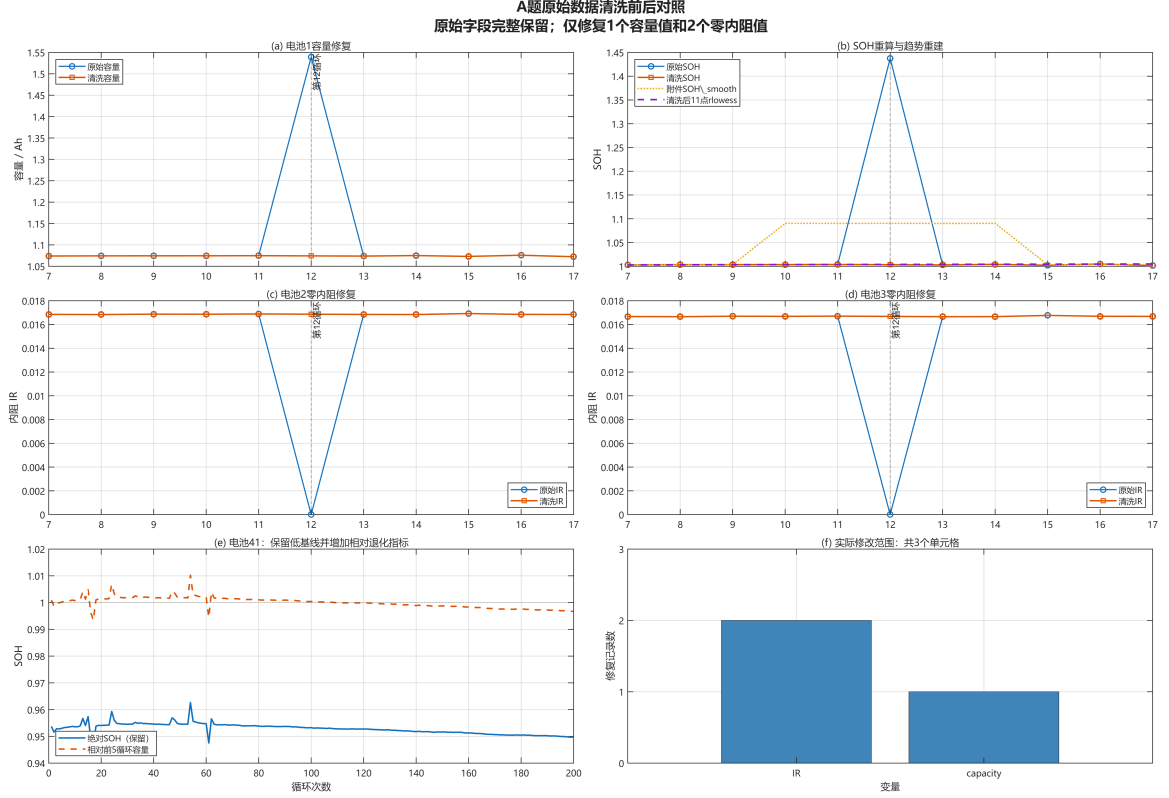


图 1 数据清洗前后对照（容量尖峰与零内阻修复）

电池 41 存在初始 SOH 基线偏低（约 0.953），判断为批次/定义差异而非单点异常，予以保留并增加相对 SOH 辅助指标；策略 $80PER_3_6C$ 的 C_1 缺失（单阶段策略），不猜测填补，仅进入类别比较。清洗后保留全部 49 块电池、9350 条记录。

六、问题一模型的建立与求解

6.1 两阶段函数型曲线模型

将每块电池的 SOH 视为关于循环数的函数曲线，令 $x = t/200$ ，以三次样条基函数

$$\mathbf{B}(x) = [1, x, x^2, x^3, (x - 0.25)_+^3, (x - 0.50)_+^3, (x - 0.75)_+^3] \quad (1)$$

建立两阶段函数型曲线主模型：先对每块电池的 SOH 曲线做三次样条岭平滑，再在策略内等权平均电池系数得到策略级曲线，即

$$\text{SOH}_{i,t} = \mathbf{B}(x_t)^\top (\gamma + \gamma_{p(i)}) + \varepsilon_{i,t} \quad (2)$$

其中 γ 为总体退化系数， $\gamma_{p(i)}$ 为策略 $p(i)$ 的偏离，按每块电池等权而非每条循环记录等权。经扩展超参数网格（含 λ_{random} 与 λ_{curve} 两个惩罚项）留一验证，最终选中 $\lambda_{\text{curve}} = 0$ ，即退化为无正惩罚的函数型曲线解。以样条混合（`spline_mixed`，带电池随机截距与随

机斜率)与多项式混合 (polynomial_mixed) 为候选对照。模型用 40 块完整电池按整块电池留一验证 (LOBO) 评价, 策略级指标以整块电池 bootstrap 估计标准误与区间。主模型训练残差的一阶自相关约为 0.71, 说明平滑后仍存在循环内序列相关, 故不确定性统一用整块电池 bootstrap 处理。

6.2 模型选择

三个候选模型的 LOBO RMSE 极其接近 (表 3), 两阶段函数型曲线以 0.004325 略优, 选为主模型; 且三模型对第 200 循环 SOH 的策略排序一致, 说明排序稳健。

表 3 问题一候选模型比较 (LOBO)

模型	平均电池 RMSE	平均电池 MAE	最差策略 RMSE	选定
polynomial_mixed	0.004394	0.004071	0.018335	否
spline_mixed	0.004342	0.004036	0.018332	否
functional_curve	0.004325	0.004026	0.018332	是

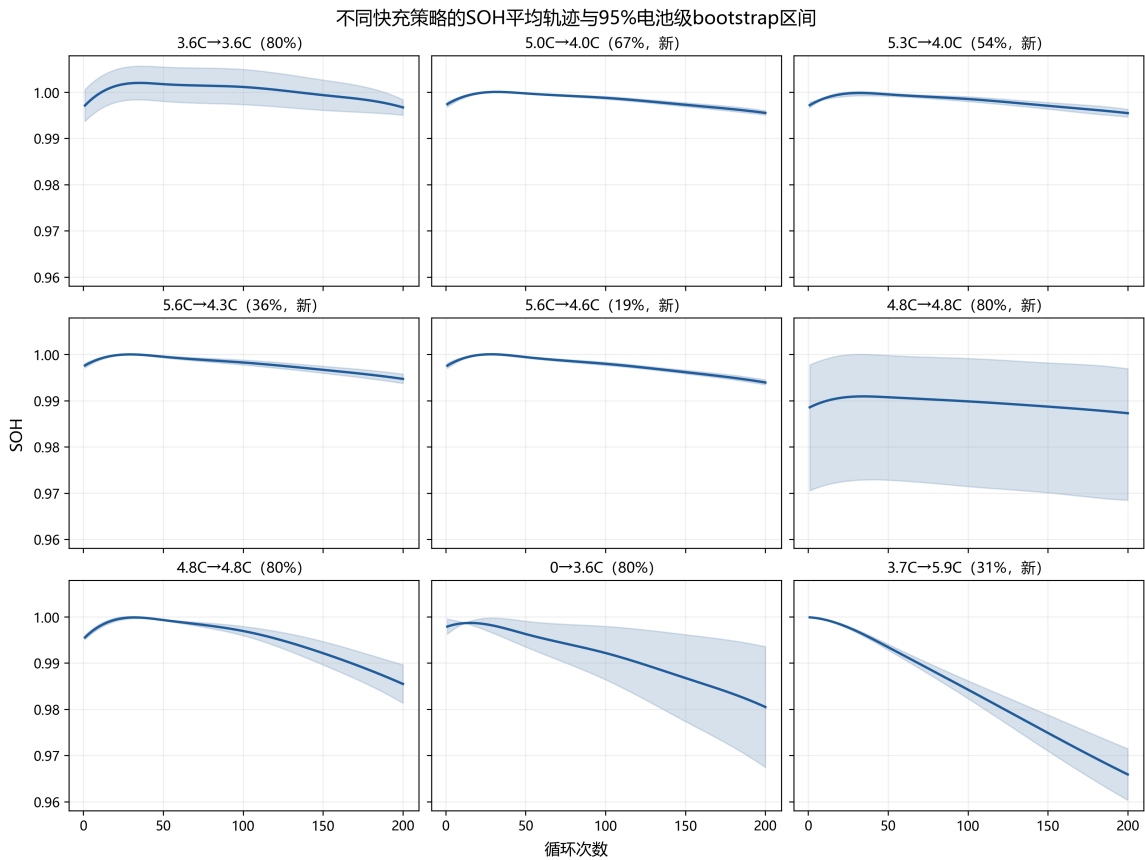


图 2 各策略 SOH 曲线 (主模型拟合)

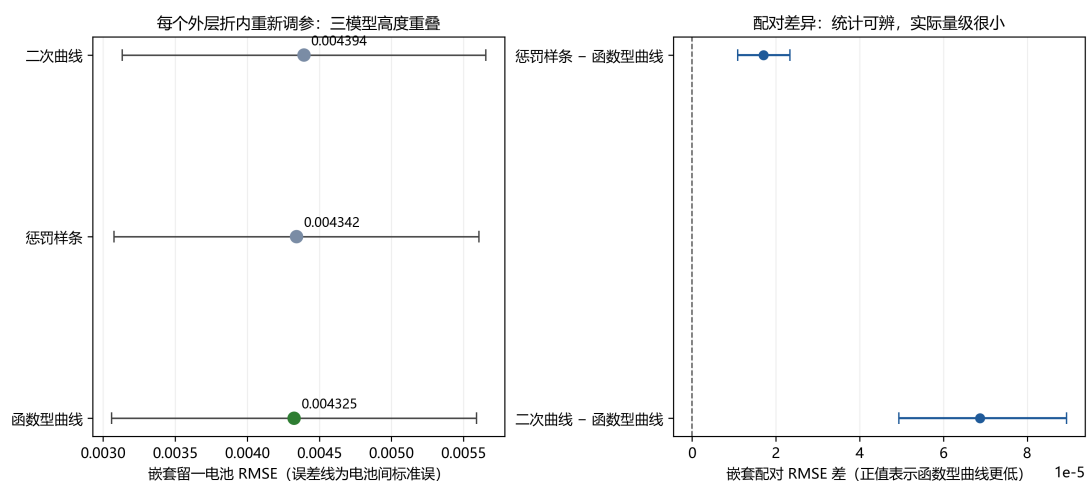


图 3 三候选模型 LOBO 误差对比

6.3 策略寿命分布与典型策略识别

由于数据无 80% 终点，以第 200 循环 SOH (SOH200) 与前 200 循环 SOH 损失两个指标共同描述寿命表现。9 种策略的完整估计见表 4，SOH200 分布在 0.9659 ~ 0.9967 之间，极差约 3.1 个百分点。

表 4 各策略寿命代理指标（按 SOH200 降序）

排名	策略	SOH200	前 200 循环损失	充电时间/min
1	3_6C-80PER_3_6C	0.9967	0.0004	13.388
2	5C_67PER_4C (新)	0.9955	0.0019	10.159
3	5_3C_54PER_4C (新)	0.9954	0.0017	10.158
4	5_6C_36PER_4_3C (新)	0.9947	0.0029	10.215
5	5_6C_19PER_4_6C (新)	0.9939	0.0036	10.250
6	4_8C_80PER_4_8C (新)	0.9873	0.0013	11.160
7	4_8C_80PER_4_8C	0.9855	0.0101	13.082
8	80PER_3_6C	0.9805	0.0174	14.280
9	3_7C_31PER_5_9C (新)	0.9659	0.0340	10.852

典型长寿命策略为 3_6C-80PER_3_6C、5C_67PER_4C 与 5_3C_54PER_4C，三者进入 Top 3 的 bootstrap 概率分别为 90.2%、85.4% 与 78.2%（前两者达到 80% 稳定性标准）。典型短寿命策略为 3_7C_31PER_5_9C、80PER_3_6C 与 4_8C_80PER_4_8C，其中 3_7C_31PER_5_9C 进入 Bottom 3 的概率为 100%，短寿命特征最稳定。

6.4 两两比较与寿命终点边界

精确置换检验配合 Holm 校正后，36 组 SOH200 两两比较中没有确证显著的策略对；但另有 20 组未校正 bootstrap 区间不跨零。论文并列报告两者，既不把前者解释为九种策略完全等效，也不把后者当作多重比较后的确证差异——每策略仅 2–7 块完整电池时两两检验功效有限。此外，49 块电池均未观测到 80% SOH 终点，局部线性外推的 L_{80} 只能作为未验证的外推代理，不作为寿命标签。

6.5 长寿命与短寿命策略的差异解释

典型长寿命策略的共同特征是限制了中高 SOC 区间的充电倍率：5C_67PER_4C 在 67% SOC 后降至 4C，5_3C_54PER_4C 在 54% SOC 后降至 4C，3_6C 则在主要快充区间保持较温和的 3.6C；而典型短寿命策略 3_7C_31PER_5_9C 在 31% SOC 即切换至 5.9C，使电池在 31%–80% SOC 的较宽区间承受高倍率，其充电时间仅 10.85 min 却对应最低的 SOH200 (0.9659)。持续的中高 SOC 高倍率可能增强电化学反应极化与欧姆发热，增加活性锂损失、界面副反应与析锂风险^[4]。

值得注意的是，结果不显示“充电时间越长寿命越好”的单调关系：5C_67PER_4C 与 5_3C_54PER_4C 的充电时间约 10.16 min，同时保持较高 SOH200，反而快于更长充电时间但更低 SOH 的 80PER_3_6C (14.28 min)。这说明高倍率出现的 SOC 区间及持续范围，比单一倍率大小或充电时间更能解释策略间寿命差异，与相同平均倍率下充电协议仍可产生不同寿命的文献观察一致^[11]。此外，4_8C_80PER_4_8C (新结构) 受电池 41 初始 SOH 基线影响较大，改用相对 SOH 或排除电池 41 后其排名上升，故不将全部 4.8C 策略归入短寿命组。更一般的敏感性分析显示：基线校正与剔除电池 41 会使中上游策略的名次发生交换，因此最可靠的结论是长/短寿命代理组本身，而非组内的精确名次。上述机制解释来自数据关联与电化学生物学文献，不构成独立随机实验下的因果结论。

七、问题二模型的建立与求解

7.1 低维应力坐标

设 $q = Q_1/100$ ，两阶段倍率函数为

$$c(s) = \begin{cases} C_1, & 0 \leq s \leq q, \\ C_2, & q < s \leq 0.8, \end{cases} \quad (3)$$

据此定义理想恒流充电时间、总电流应力与高 SOC 加权应力：

$$T_0 = 60 \left(\frac{q}{C_1} + \frac{0.8 - q}{C_2} \right), \quad J = qC_1 + (0.8 - q)C_2, \quad H = \frac{1}{2} [C_1 q^2 + C_2 (0.8^2 - q^2)]. \quad (4)$$

J 描述主要充电区间的总体倍率暴露， H 对高 SOC 区间赋予更高权重。二者相关系数为 0.87， T_0 与 J 相关系数为 -0.984，三者不能同时进入正式模型。此外定义”高于阈值 s_0 的 SOC 区间应力”

$$J_{\text{high},s_0} = C_1 \max(q - s_0, 0) + C_2 (0.8 - \max(q, s_0)), \quad s_0 \in \{0.5, 0.6, 0.7\}, \quad (5)$$

用于隔离中高 SOC 高倍率暴露对退化的作用。参数设计审计显示：7 个参数完整的非基准策略的理想恒流时间 T_0 均约 9.99 ~ 10.01 min（仅 3_6C 基准为 13.33 min），即实验设计基本固定了名义恒流时间；而实测充电时间与 T_0 的差值达 0.14 ~ 3.08 min，反映恒压尾段与结构/批次的影响，不能只由 C_1, Q_1, C_2 精确决定。

7.2 约束退化混合模型

以第 151–200 循环的末段退化率（对数尺度）为响应，建立

$$y_{it} = a_i - r_i x_t - k x_t^2 + \varepsilon_{it}, \quad x_t = t/200, \quad (6)$$

$$\log r_i = \beta_0 + \beta_J \tilde{J}_{p(i)} + \beta_H \tilde{H}_{p(i)} + \beta_S S_{p(i)} + u_i \quad (7)$$

其中 $r_i > 0$ 、 $k \geq 0$ 保证平均 SOH 单调下降， u_i 为同策略电池差异， S 为”新结构/数据批次”联合标签（不作纯结构因果解释）。模型选择使用留一策略验证、AICc 与系数稳定性。

7.3 显著性检验的诊断属性

对策略间末段退化率做全局置换检验： $F = 12.50$ 、尾部比例 5×10^{-5} （20000 次置换）。**该值需谨慎解读：**由于协议组固定且各策略样本量、方差不等，全电池标签可交换假设并不严格成立，因此该尾部比例是**诊断性而非确证性 p 值**。在单暴露模型中，高 SOC 分位应力（ $J_{\text{high},50/60/70}$ ）均与退化方向一致，其中选定模型 $J_{\text{high},70}$ （SOC 高于 70% 区间的倍率暴露）的标准化系数为 0.0039（对相对损失为正、对 SOH200 为负），单暴露未校正尾部比例为 0.019；但经四个暴露量的选择校正后尾部比例为 0.065，且删除极端策略后常数模型胜出。2000 次整块电池 bootstrap 进一步显示， J_{high} 系数 100% 为正（方向稳健），而 H 相对常数基线的拟合改善为负，支持”高 SOC 区间应力而非总体应力是更一致的退化解释量”这一判断。综上， J_{high} 与退化的关联方向稳健，但不能据此宣称 SOC 阈值或参数的独立因果效应。

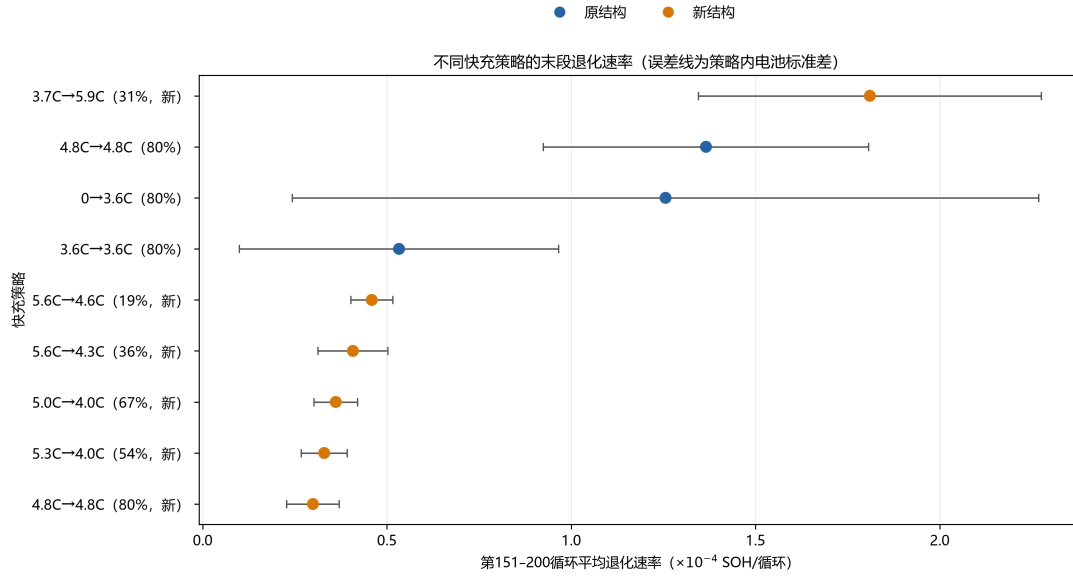


图 4 各策略末段（151–200 循环）退化率

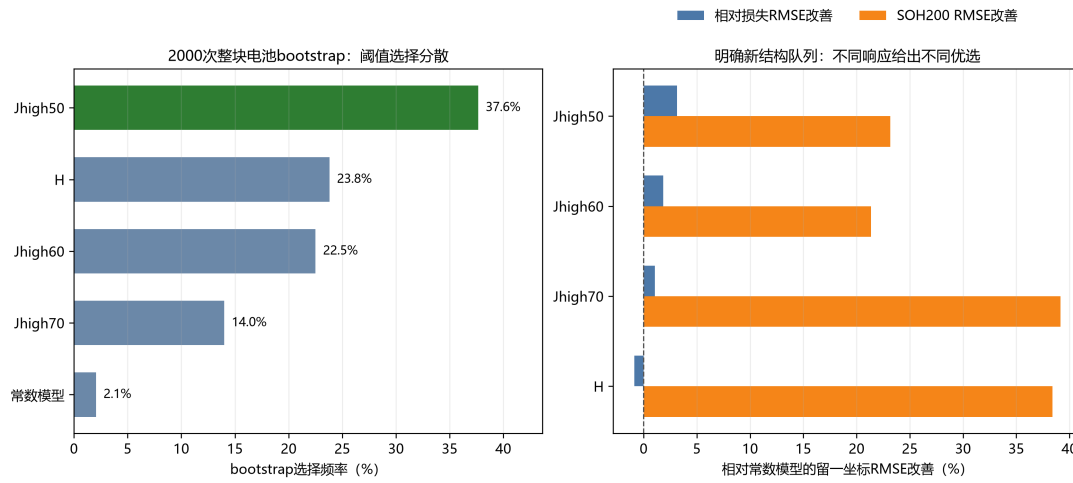


图 5 模型系数稳定性与选择校正

7.4 结构与批次混杂

两个具有相同 (4.8, 0.8, 4.8) 参数的策略分属旧/新结构，末段退化率存在差异；但结构、数据批次与策略参数完全混杂，无法将此差异干净地归因于”结构”。

7.5 小问结论

综上，回答五个小问：（1）策略间末段退化存在全局差异（诊断性尾部比例 5×10^{-5} ）；（2–3） C_1, Q_1, C_2 与退化的关联方向一致（高 SOC 高倍率加剧退化），但受稀疏混杂设计限制，不能识别三个参数的独立因果效应；（4）不同 SOC 区间高倍率的差异化作用方向与机理预期一致（高 SOC 分位应力 J_{high} 是更一致的退化解释量），但证据

强度不足以形成因果断言；(5) 结论的机制可解释为高 SOC 下高倍率促进锂析出与极化加剧，其局限性在于设计混杂与样本量不足^[4]。

八、问题三模型的建立与求解

8.1 特征提取

从每块电池前 L 循环的观测提取三类特征（表 5）：**(1) 轨迹形态特征**——采样 SOH 点位、全区间线性斜率、多窗口局部斜率、近期曲率（近期斜率与前期斜率之差）、线性拟合残差标准差、近期变化范围；**(2) 运行状态动态特征**——内阻、平均温度、充电时间的均值、近期均值与斜率；**(3) 策略参数与应力坐标**—— C_1, q, C_2 、 C_1 缺失标记，以及 $J, H, J_{\text{high},50/60/70}$ 。特征在每折内部用训练电池的中位数填充缺失、标准化。

表 5 问题三特征集

类别	特征
轨迹形态	采样 SOH 点位、全区间斜率、多窗口局部斜率、曲率、残差标准差、近期范围
运行状态	内阻、平均温度、充电时间的均值、近期均值与斜率（共 9 维）
策略参数	C_1, q, C_2 、 C_1 缺失标记、 $J, H, J_{\text{high},50/60/70}$

8.2 候选模型与部署选择

比较六类候选模型，主模型 $P1_linear$ （线性外推）用前 L 循环拟合线性趋势并外推未来 SOH。因部署时固定提供 150 个观测循环，按 $L = 150$ 外层留一电池冻结 $P1_linear$ ：其在 $L = 150$ 的策略等权 LOBO RMSE 为 0.000617（表 6）。轨迹岭模型 C_ridge 在多长度综合指标上排名第一，但该指标是将不同 L 的误差加权合成，与最终固定 $L = 150$ 的部署场景不同，故保留为多长度综合稳健性敏感性而非部署模型。六模型排名见表 6，第 2–4 名处于约 2% 并列容差内，名次不稳定。

表 6 六类候选模型 $L = 150$ 排名

排名	模型	策略等权 RMSE	部署/角色	选定
1	C_ridge (轨迹岭回归)	0.000669	多长度稳健敏感性	否
2	$D_ensemble$ (集成)	0.000701	候选	否
3	$P1_linear$ (线性外推)	0.000617	部署	是
4	$B_strategy$ (策略迁移)	0.000785	候选	否
5	A_power (幂律)	0.000893	候选	否
6	$P0_persistence$ (持久化)	0.002400	候选	否

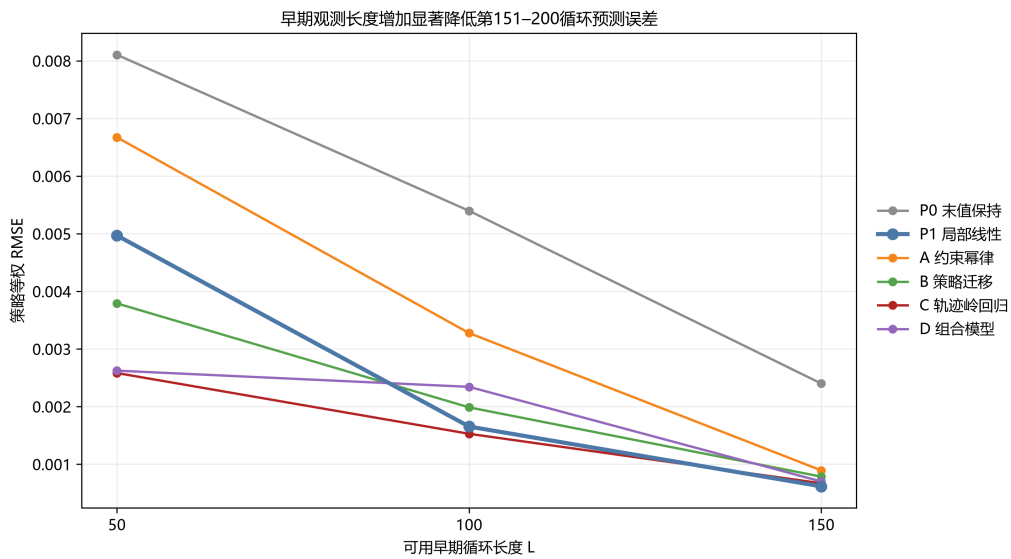


图 6 六候选模型随早期数据长度的 LOBO RMSE

$L = 150$ 特征消融显示 (表 7): 仅用动态轨迹特征的 RMSE 0.000650 略优于完整模型, 说明策略参数特征在该窗口不提供增量精度。预测区间由冻结模型族在 40 块训练电池上的外层交叉拟合绝对残差校准, 以 95% 目标边际覆盖率取第 39/40 顺序统计量, 为偏保守的逐循环点区间; 同一残差也参与选族, 可能受赢家偏差影响, 不构成独立测试覆盖率。

表 7 $L = 150$ 特征消融 (策略等权 RMSE)

特征组合	RMSE
仅动态轨迹特征	0.000650
动态 + 策略参数	0.000686
完整 (含策略标签)	0.000669

8.3 测试电池预测

9 块测试电池的 $P1_linear$ 预测 SOH200 与保守点区间见表 8。区间为逐循环点区间，不构成整条轨迹联合区间或独立测试覆盖率。

表 8 9 块测试电池预测 ($P1_linear$)

电池	策略	预测 SOH200	保守点区间	情景 T_{80}
2	3_6C-80PER_3_6C	0.994184	[0.992014, 0.996353]	1249.4
9	4_8C_80PER_4_8C	0.982823	[0.980654, 0.984993]	706.0
16	3_7C_31PER_5_9C	0.961868	[0.959699, 0.964038]	—
25	4_8C_80PER_4_8C (新)	0.996390	[0.994221, 0.998560]	—

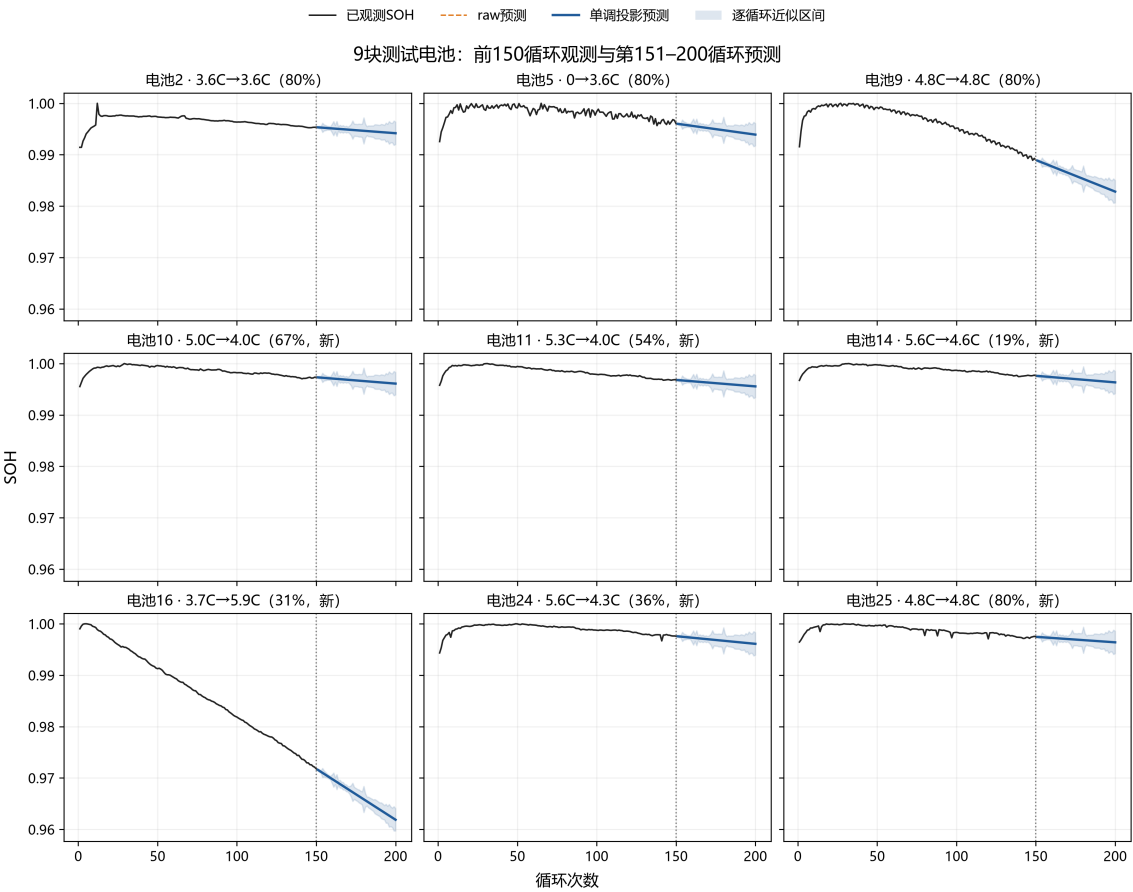


图 7 9 块测试电池 151–200 循环预测（实线单调投影、阴影保守点区间）

8.4 80% 寿命外推的边界

80% 寿命终点 (T_{80}) 对拟合起点与 raw/projected 口径高度敏感 (图 8)，有限情景值范围为 **706.0–4899.3** 循环。需特别说明：表中的拼接幂律情景是”真实 1–150 循环与

线性预测 151–200 循环拼接后另行拟合幂律曲线”得到的，不能解释为部署线性模型自身的寿命终点（后者单独列为 native 线性外推状态，部分电池超过 5000 循环）。由于 49 块电池均未观测到 80% SOH， T_{80} 只能作为模型情景交点，不具有实证寿命精度，论文将其与短期 LOBO 误差分开陈述。

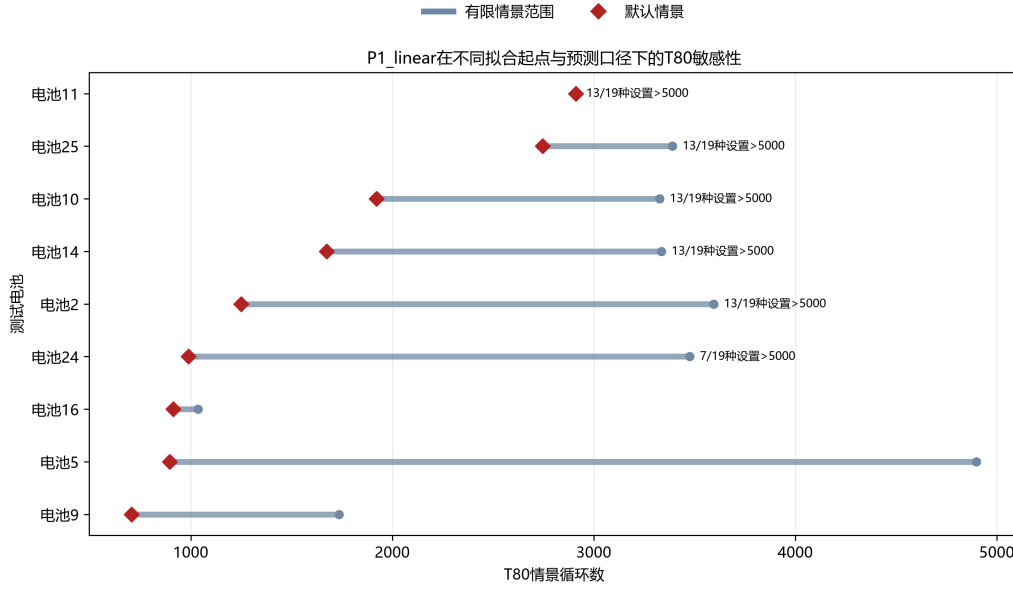


图 8 T_{80} 情景外推敏感性（红菱形为默认情景）

九、问题四模型的建立与求解

9.1 离散 Pareto 主模型 (M0)

对策略 p 定义充电时间与退化量

$$T_p = \text{mean}_{i \in p}(\text{chargetime}_i), \quad D_p = \text{mean}_{i \in p} \left(1 - \text{SOH}_{i,200}/b_i \right), \quad (8)$$

其中 b_i 为电池基线容量。若不存在策略 r 使 $T_r \leq T_p$ 且 $D_r \leq D_p$ （至少一项严格），则 p 位于 Pareto 前沿。时间与退化用整块电池 bootstrap 量化不确定性。理论恒流时间 T_0 仅作解释量，实测时间含恒压尾段与结构/批次影响。

9.2 Pareto 前沿与推荐

点估计 Pareto 前沿含 3_6C 基准、4_8C 新结构与 5_3C_54PER_4C（表 9），bootstrap 显示前沿成员不稳定。5_3C_54PER_4C 在点估计上同时更快且退化更低，是点 Pareto 快速推荐；5C_67PER_4C 严格被支配，但二者时间差与退化差的整块电池 bootstrap 区间均跨零（退化差区间 $[-0.0015, +0.0012]$ ），且 5_3C 退化更低的概率不足 0.95，因此它保留为非前沿近似并列敏感性项，而不是共同主推荐。

表 9 点估计 Pareto 前沿（40 块完整电池）

策略	n	时间/min	200 循环退化	末段衰减率	Pareto 频率
3_6C-80PER_3_6C	2	13.388	0.000520	0.0000500	0.746
4_8C_80PER_4_8C（新）	5	11.039	0.001436	0.0000277	0.518
5_3C_54PER_4C	7	10.158	0.001800	0.0000315	0.890

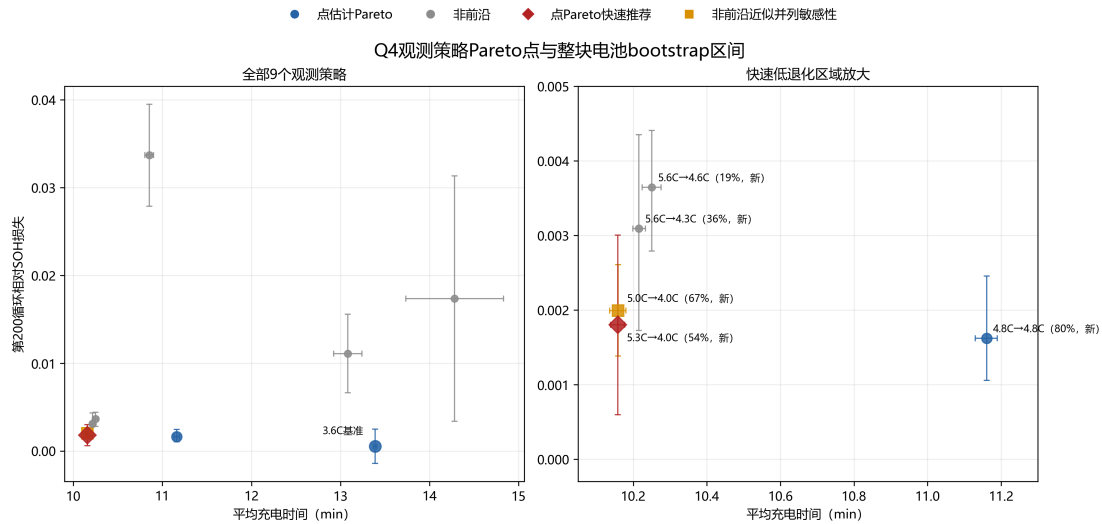


图 9 充电时间-退化 Pareto 前沿与 bootstrap 不确定性

表 10 推荐策略与典型策略对比

策略	时间/min	200 循环退化	较长寿命时间差	较长寿命退化差
3_6C-80PER_3_6C（典型长寿命）	13.388	0.000520	0	0
5_3C_54PER_4C（推荐）	10.158	0.001800	-3.230	+0.001280
5C_67PER_4C（近似并列敏感）	10.159	0.001993	-3.229	+0.001472
3_7C_31PER_5_9C（典型短寿命）	10.852	0.033684	-2.536	+0.033164

推荐 5_3C_54PER_4C ($C_1 = 5.3, Q_1 = 54\%, C_2 = 4.0$): 若应用要求优先缩短时间并接受约 0.13% 的 200 循环相对退化, 该策略是点 Pareto 快速推荐。相对典型长寿命基准缩短约 24% 时间、退化仅增约 0.13 个百分点; 相对典型短寿命策略更快且退化低约 0.032。其机理是低 SOC 段使用较高倍率、较早切换至 4C, 形成观测数据支持的时间-退化折中。注意它在时间上属于四策略近似并列组 (最快四个新结构策略仅相差不足 0.03 s), 不能宣称显著更快。

末段斜率口径揭示一处反转:3_6C 基准的 200 循环累计退化点估计最低(0.000520), 但其末段 (151-200) 每循环衰减率 0.0000500 反而高于 4_8C 新结构 (0.0000277) 与

5_3C (0.0000315)，因此在末段斜率前沿上被二者支配。只有 4_8C 新结构与 5_3C 在 SOH200 与末段斜率两个口径下都位于前沿；而 3_6C 的累计退化均值仅来自 $n = 2$ 样本且策略内标准差高于均值本身，不宜对“最低退化”作过强断言。

9.3 条件约束决策与连续响应面淘汰

除单点推荐外，本文还给出“给定退化容忍度 $D_{\max}$ 下选择最短时间”的条件决策：当 $D_{\max} = 0.0005$ 时以 74% 概率无可行策略、3_6C 基准被选频率 24.4%；当 $D_{\max} = 0.0017$ 时 5_3C 被选频率 42.4%、4_8C 新结构 28.9%。这些容忍度仅为展示决策规则的示例场景，不是工程安全标准。权重扫描（min-max 标准化）仅作偏好敏感性诊断：使用全部 9 个观测策略（含被支配点）归一化时 $\lambda = 0.1$ 即选择 5.3C；而仅用 Pareto 点归一化时，要到 $\lambda = 0.6$ 才从 3.6C 切换到 5.3C。该差异说明权重结论依赖标准化集合，不能把任一加权表当成无条件主推荐，正式决策以 Pareto 前沿、退化约束与 bootstrap 稳定性为准。

受限连续响应面（单 J 岭模型）在 oracle 坐标压力测试中平均 RMSE 为 0.015827，7 折中 6 折未优于常数基线。进一步审计显示，最差的 3.6C 留一折位于训练 J 范围之外，产生负退化预测并贡献总平方误差的 72.2%；剔除该折后 M1 平均 RMSE 仍为 0.009652，略差于常数基线 0.009324，故失败方向不依赖该折。据此连续三参数优化被正式淘汰，正式答案退回离散 Pareto；但这只否定当前单 J 岭代理，不证明所有连续代理或后续新增实验均无效。

9.4 合理性、适用范围与外推风险

推荐合理性来自：位于点估计 Pareto 前沿、bootstrap Pareto 频率最高（0.890）、两种时间口径下前沿不变、SOH200 与末段斜率结论方向一致。结论严格限定在 9 个已观测策略范围内，不宣称 C_1, Q_1, C_2 的独立因果最优值，不外推到未观测倍率、SOC 区间或新电芯结构^[4,5]。

十、模型检验与灵敏度分析

10.1 清洗规则敏感性

固定 3 处修复但改变局部趋势窗口（7/11/15 点）后，策略排序不变；保留异常原始值仅影响个别策略指标。

10.2 响应口径敏感性

Q1 以 SOH200、前 200 循环损失、曲线面积与末段斜率多口径检验，策略排序高度一致；Q4 的退化代理允许为负（受基线噪声影响），如 3.6C 策略退化均值为 0.000520、

bootstrap 区间下界为负，已接近测量噪声地板。

10.3 时间口径敏感性

Q4 采用 40 块完整电池逐循环充电时间均值为主口径，官方汇总口径作敏感性审计；两口径在部分策略差异明显（如旧结构 4.8C 为 10.45 与 13.08 min），但 Pareto 前沿与推荐策略不变。

10.4 模型选择敏感性

Q3 部署模型对选择标准敏感： $L = 150$ 单点最优为 $P1_linear$ ，多长度综合最优为 C_ridge ，论文明确区分部署模型与稳健性敏感性，避免把综合指标误当部署依据。

十一、模型评价与推广

11.1 模型优点

- 数据清洗最小干预、全程审计留痕，修复决策与人工核对一致；
- 始终以整块电池为推断单位，正确处理重复测量与聚类重采样；
- 多模型互证，并用置换检验、选择校正、bootstrap 量化不确定性，且明确区分诊断性结论与确证性结论；
- 诚实地报告了设计混杂导致的因果识别边界，不强行宣称参数独立效应。

11.2 模型不足

- 9 种策略的稀疏混杂设计使 C_1, Q_1, C_2 独立因果效应不可识别；
- 无真实 80% SOH 终点，寿命终点均为外推；
- 2 电池策略（如 3.6C）的分布组合极少，bootstrap 只能作粗粒度证据。

11.3 推广方向

可引入层次贝叶斯模型刻画策略总体与电池个体差异并部分汇聚^[3,9]，在闭环优化框架中结合贝叶斯优化与不确定性进行实验设计^[2]，并针对末段斜率反映的退化反转做更细致的机理分析^[5]。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于【代码调试、语言润色、数据处理与建模方案讨论】，详细使用情况见支撑材料。

参考文献

- [1] Severson K A, Attia P M, Jin N, et al. Data-driven prediction of battery cycle life before capacity degradation[J]. Nature Energy, 2019, 4(5): 383-391.
- [2] Attia P M, Grover A, Jin N, et al. Closed-loop optimization of fast-charging protocols for batteries with machine learning[J]. Nature, 2020, 578(7795): 397-402.
- [3] Jiang B, Gent W E, Mohr F, et al. Bayesian learning for rapid prediction of lithium-ion battery-cycling protocols[J]. Joule, 2021, 5(12): 3187-3203.
- [4] Keil P, Jossen A. Charging protocols for lithium-ion batteries and their impact on cycle life[J]. Journal of Energy Storage, 2016, 6: 125-141.
- [5] Schuster S F, Bach T C, Fleder E, et al. Nonlinear aging characteristics of lithium-ion cells under different operational conditions[J]. Journal of Energy Storage, 2015, 1: 44-53.
- [6] Saxena S, Xing Y, Kwon D, et al. Accelerated degradation model for C-rate loading of lithium-ion batteries[J]. International Journal of Electrical Power & Energy Systems, 2019, 107: 438-445.
- [7] Richardson R R, Osborne M A, Howey D A. Gaussian process regression for forecasting battery state of health[J]. Journal of Power Sources, 2017, 357: 209-219.
- [8] Liu K, Hu X, Wei Z, et al. Modified Gaussian process regression models for cyclic capacity prediction of lithium-ion batteries[J]. IEEE Transactions on Transportation Electrification, 2019, 5(4): 1225-1236.
- [9] Zhou Z, Howey D A. Bayesian hierarchical modelling for battery lifetime early prediction[J]. IFAC-PapersOnLine, 2023, 56(2): 6117-6123.
- [10] Geslin A, van Vlijmen B, Cui X, et al. Selecting the appropriate features in battery lifetime predictions[J]. Joule, 2023, 7(9): 1956-1965.
- [11] Zhang S S. The effect of the charging protocol on the cycle life of a Li-ion battery[J]. Journal of Power Sources, 2006, 161(2): 1385-1391.

附录 A 支撑材料文件列表

表 11 支撑材料文件列表

类别	路径	说明
数据	data/processed/q1_cleaned/*.csv	清洗后摘要表与循环表
结果	result/q1/paper/*.csv	问题一结果与结论
结果	result/q2/03_formal_validation/	问题二正式验证
结果	result/q3/03_final_predictions/	问题三最终预测
结果	result/q4/02_full_validation/	问题四全量验证与推荐
代码	src/q1_models/ 等	各问模型核心源码
代码	scripts/q1-q4/	各问实验入口脚本

附录 B 核心算法代码

以下为核心模型源码（完整可运行代码见支撑材料，运行顺序见各 scripts 入口）。

```

1 """Numerical core for the three comparable Question 1 curve models."""
2
3 from __future__ import annotations
4
5 from dataclasses import dataclass
6 from typing import Iterable
7
8 import numpy as np
9 import pandas as pd
10
11 MODEL_POLYNOMIAL = "polynomial_mixed"
12 MODEL_SPLINE = "spline_mixed"
13 MODEL_FUNCTIONAL = "functional_ridge"
14 MODEL_TYPES = (MODEL_POLYNOMIAL, MODEL_SPLINE, MODEL_FUNCTIONAL)
15
16
17 @dataclass(frozen=True)
18 class ModelConfig:
19     lambda_random: float = 0.1
20     lambda_curve: float = 0.01
21
22
23 @dataclass
24 class PopulationCurveModel:
25     model_type: str
26     config: ModelConfig
27     policy_names: tuple[str, ...]
28     fixed_coef: np.ndarray
29     battery_ids: np.ndarray
30     random_coef: np.ndarray | None = None
31     cell_coef: np.ndarray | None = None
32     cell_policy: np.ndarray | None = None
33
34     def predict(self, policy: str | Iterable[str], cycle: Iterable[float]) -> np.ndarray:
35         cycles = np.asarray(cycle, dtype=float).reshape(-1)
36         policies = np.asarray([policy] * len(cycles) if isinstance(policy, str) else policy, dtype=str)
37         if len(policies) != len(cycles):
38             raise ValueError("policy and cycle must have the same length")
39         basis = make_basis(self.model_type, cycles)
40         output = np.full(len(cycles), np.nan, dtype=float)
41         policy_to_index = {name: i for i, name in enumerate(self.policy_names)}
42         for name in np.unique(policies):
43             if name not in policy_to_index:
44                 raise KeyError(f"policy absent from training data: {name}")
45             use = policies == name
46             output[use] = basis[use] @ self.fixed_coef[policy_to_index[name]]
47         return output
48
49
50 def make_basis(model_type: str, cycle: Iterable[float]) -> np.ndarray:
51     """Return the explicitly specified cycle basis, using x=t/200."""
52     x = np.asarray(cycle, dtype=float).reshape(-1) / 200.0
53     if model_type == MODEL_POLYNOMIAL:
54         return np.column_stack((np.ones_like(x), x, x**2))
55     if model_type in (MODEL_SPLINE, MODEL_FUNCTIONAL):
56         columns = [np.ones_like(x), x, x**2, x**3]
57         columns.extend(np.maximum(x - knot, 0.0) ** 3 for knot in (0.25, 0.50, 0.75))
58         return np.column_stack(columns)
59     raise ValueError(f"unknown model type: {model_type}")

```

```

61
62
63 def candidate_configs(model_type: str) -> list[ModelConfig]:
64     if model_type == MODEL_POLYNOMIAL:
65         return [ModelConfig(value, 0.0) for value in (0.01, 0.1, 1.0, 10.0)]
66     if model_type == MODEL_SPLINE:
67         return [
68             ModelConfig(random_penalty, curve_penalty)
69             for random_penalty in (0.03, 0.3, 3.0)
70             for curve_penalty in (0.001, 0.01, 0.1, 1.0)
71         ]
72     if model_type == MODEL_FUNCTIONAL:
73         return [ModelConfig(0.0, value) for value in (0.0001, 0.001, 0.01, 0.1, 1.0)]
74     raise ValueError(f"unknown model type: {model_type}")
75
76
77 def fit_population_model(data: pd.DataFrame, model_type: str, config: ModelConfig) -> PopulationCurveModel:
78     """Fit a strategy population curve with equal inferential units at battery level.
79
80     The two mixed candidates minimize
81     
$$||y - F\beta - Zb||^2 + \lambda_{\text{curve}} ||P\beta||^2 + \lambda_{\text{random}} ||b||^2,$$

82     where Z contains a random intercept and slope for each battery. The
83     functional candidate first smooths each battery, then averages cell
84     coefficients within strategy so batteries receive equal weight.
85     """
86
87     required = {"battery_id", "cycle", "policy", "SOH_clean"}
88     missing = required.difference(data.columns)
89     if missing:
90         raise ValueError(f"missing columns: {sorted(missing)}")
91     frame = data.loc[np.isfinite(data["SOH_clean"]) & np.isfinite(data["cycle"])]
92     frame["policy"] = frame["policy"].astype(str)
93     policy_names = tuple(pd.unique(frame["policy"]).tolist())
94     battery_ids = pd.unique(frame["battery_id"]).astype(int)
95     policy_index = {name: i for i, name in enumerate(policy_names)}
96     battery_index = {int(value): i for i, value in enumerate(battery_ids)}
97     basis = make_basis(model_type, frame["cycle"].to_numpy())
98     n_rows, n_basis = basis.shape
99
100     if model_type in (MODEL_POLYNOMIAL, MODEL_SPLINE):
101         fixed = np.zeros((n_rows, len(policy_names) * n_basis), dtype=float)
102         pid_x = frame["policy"].map(policy_index).to_numpy(dtype=int)
103         rows = np.arange(n_rows)
104         for j in range(n_basis):
105             fixed[rows, pid_x * n_basis + j] = basis[:, j]
106
107         random = np.zeros((n_rows, 2 * len(battery_ids)), dtype=float)
108         bid_x = frame["battery_id"].map(battery_index).to_numpy(dtype=int)
109         x = frame["cycle"].to_numpy(dtype=float) / 200.0
110         random[rows, 2 * bid_x] = 1.0
111         random[rows, 2 * bid_x + 1] = x
112         design = np.column_stack((fixed, random))
113
114         penalty = np.zeros(design.shape[1], dtype=float)
115         if model_type == MODEL_SPLINE:
116             for p in range(len(policy_names)):
117                 start = p * n_basis
118                 penalty[start + 2 : start + n_basis] = config.lambda_curve
119             penalty[len(policy_names) * n_basis :] = config.lambda_random
120             coefficient = _penalized_solve(design, frame["SOH_clean"].to_numpy(), penalty)
121             fixed_coef = coefficient[: len(policy_names) * n_basis].reshape(len(policy_names), n_basis)
122             random_coef = coefficient[len(policy_names) * n_basis :].reshape(len(battery_ids), 2)
123             return PopulationCurveModel(
124                 model_type, config, policy_names, fixed_coef, battery_ids, random_coef=random_coef
125             )
126
127     if model_type == MODEL_FUNCTIONAL:
128         cell_coef = np.empty((len(battery_ids), n_basis), dtype=float)
129         cell_policy = np.empty(len(battery_ids), dtype=object)
130         penalty = np.zeros(n_basis, dtype=float)
131         penalty[2:] = config.lambda_curve
132         for i, battery_id in enumerate(battery_ids):
133             use = frame["battery_id"].to_numpy() == battery_id
134             cell_coef[i] = _penalized_solve(basis[use], frame.loc[use, "SOH_clean"].to_numpy(), penalty)
135             cell_policy[i] = frame.loc[use, "policy"].iloc[0]
136         fixed_coef = np.vstack([cell_coef[cell_policy == name].mean(axis=0) for name in policy_names])
137         return PopulationCurveModel(
138             model_type,
139             config,
140             policy_names,
141             fixed_coef,
142             battery_ids,
143             cell_coef=cell_coef,
144             cell_policy=cell_policy,
145         )
146     raise ValueError(f"unknown model type: {model_type}")
147
148
149 def _penalized_solve(design: np.ndarray, response: np.ndarray, penalty: np.ndarray) -> np.ndarray:
150     lhs = design.T @ design
151     scale = max(float(np.trace(lhs)) / max(lhs.shape[0], 1), 1.0)
152     lhs.flat[:: lhs.shape[0] + 1] += penalty + 1e-12 * scale
153     rhs = design.T @ response
154     try:
155         return np.linalg.solve(lhs, rhs)
156     except np.linalg.LinAlgError:
157         return np.linalg.lstsq(lhs, rhs, rcond=1e-12)[0]

```

```

1 """Shared numerical core for Question 2 smoke tests.
2
3 The strategy parameter coordinate, rather than a cycle row or a battery, is the
4 independent design unit. All parameter-response regressions therefore average
5 within strategy first and validate by leaving an entire parameter coordinate out.
6 """
7
8 from __future__ import annotations
9
10 from dataclasses import dataclass
11 from pathlib import Path
12
13 import numpy as np
14 import pandas as pd
15
16
17 SEED = 20260814
18 LAMBDA_GRID = (0.0, 0.01, 0.1, 1.0, 10.0, 100.0)
19 CHECKPOINTS = np.array([25, 50, 75, 100, 125, 150, 175, 200], dtype=int)
20
21
22 @dataclass(frozen=True)
23 class Candidate:
24     name: str
25     features: tuple[str, ...]
26     family: str = "ridge"
27     quadratic_cycle: bool = False
28
29
30 STRATEGY_CANDIDATES = (
31     Candidate("constant_mean", (), "constant"),
32     Candidate("nearest_coordinate", ("C1", "q", "C2"), "nearest"),
33     Candidate("ridge_raw_C1_q_C2", ("C1", "q", "C2")),
34     Candidate("ridge_stage_E1_E2", ("E1", "E2")),
35     Candidate("ridge_J", ("J",)),
36     Candidate("ridge_H", ("H",)),
37     Candidate("ridge_J_H", ("J", "H")),
38     Candidate("ridge_J_high50", ("J_high_50",)),
39     Candidate("ridge_J_high60", ("J_high_60",)),
40     Candidate("ridge_J_high70", ("J_high_70",)),
41 )
42
43
44 HIERARCHICAL_CANDIDATES = (
45     Candidate("hier_cycle_no_stress", (), "hierarchical", False),
46     Candidate("hier_cycle_J_linear", ("J",), "hierarchical", False),
47     Candidate("hier_cycle_H_linear", ("H",), "hierarchical", False),
48     Candidate("hier_cycle_J_H_linear", ("J", "H"), "hierarchical", False),
49     Candidate("hier_cycle_J_quadratic", ("J",), "hierarchical", True),
50 )
51
52
53 def load_clean_data(project_root: Path) -> tuple[pd.DataFrame, pd.DataFrame]:
54     cycle_path = project_root / "data" / "processed" / "q1_cleaned" / "cycle_train_clean.csv"
55     summary_path = project_root / "data" / "processed" / "q1_cleaned" / "battery_summary_clean.csv"
56     cycles = pd.read_csv(cycle_path)
57     summary = pd.read_csv(summary_path)
58     counts = cycles.groupby("battery_id", observed=True)["cycle"].nunique()
59     complete_ids = counts[counts.eq(200)].index
60     cycles = cycles[cycles["battery_id"].isin(complete_ids)].copy()
61     summary = summary[summary["battery_id"].isin(complete_ids)].copy()
62     if cycles["SOH_clean"].isna().any():
63         raise ValueError("SOH_clean contains missing values in the complete cohort")
64     return cycles, summary
65
66
67 def coordinate_id(frame: pd.DataFrame) -> pd.Series:
68     return (
69         frame["C1"].map(lambda value: f"{value:.3f}")
70         + "|"
71         + frame["q"].map(lambda value: f"{value:.3f}")
72         + "|"
73         + frame["C2"].map(lambda value: f"{value:.3f}")
74     )
75
76
77 def add_protocol_features(frame: pd.DataFrame) -> pd.DataFrame:
78     result = frame.copy()
79     result["q"] = result["Q1"] / 100.0
80     result["structure_batch"] = result["policy"].astype(str).str.contains("NEWSTRUCTURE").astype(int)
81     result["T0"] = 60.0 * (result["q"] / result["C1"] + (0.8 - result["q"]) / result["C2"])
82     result["E1"] = result["q"] * result["C1"]
83     result["E2"] = (0.8 - result["q"]) * result["C2"]
84     result["J"] = result["E1"] + result["E2"]
85     result["H"] = 0.5 * (
86         result["C1"] * result["q"] ** 2
87         + result["C2"] * (0.8 ** 2 - result["q"] ** 2)
88     )
89     for threshold in (0.5, 0.6, 0.7):
90         result[f"J_high_{int(threshold * 100)}"] = np.where(
91             threshold < result["q"],
92             result["C1"] * (result["q"] - threshold)
93             + result["C2"] * (0.8 - result["q"]),
94             result["C2"] * (0.8 - threshold),
95         )
96     result["coordinate_id"] = coordinate_id(result)
97     return result
98
99

```

```

100 def battery_degradation_summary(cycles: pd.DataFrame, battery_meta: pd.DataFrame) -> pd.DataFrame:
101     records: list[dict[str, float | int | str]] = []
102     meta = battery_meta.set_index("battery_id")
103     for battery_id, group in cycles.groupby("battery_id", observed=True):
104         group = group.sort_values("cycle")
105         baseline = float(group.loc[group["cycle"].between(1, 5), "SOH_clean"].mean())
106         end_soh = float(group.loc[group["cycle"].between(196, 200), "SOH_clean"].mean())
107         x = group["cycle"].to_numpy(dtype=float) / 200.0
108         degradation = 1.0 - group["SOH_clean"].to_numpy(dtype=float) / baseline
109         design = np.column_stack((np.ones_like(x), x, x**2))
110         coef = np.linalg.lstsq(design, degradation, rcond=None)[0]
111         row = meta.loc[battery_id]
112         records.append(
113             {
114                 "battery_id": int(battery_id),
115                 "policy": str(row["policy"]),
116                 "C1": float(row["C1"]) if np.isfinite(row["C1"]) else np.nan,
117                 "Q1": float(row["Q1"]),
118                 "C2": float(row["C2"]),
119                 "baseline_soh": baseline,
120                 "soh200": end_soh,
121                 "relative_loss200": 1.0 - end_soh / baseline,
122                 "curve_loss200": float(coef.sum()),
123                 "curve_linear": float(coef[1]),
124                 "curve_quadratic": float(coef[2]),
125                 "mean_chargetime": float(row["mean_chargetime"]),
126                 "mean_IR": float(row["mean_IR"]),
127                 "mean_Tavg": float(row["mean_Tavg"]),
128                 "flag_battery41": int(battery_id == 41),
129             }
130         )
131     return add_protocol_features(pd.DataFrame.from_records(records))
132
133
134 def strategy_summary(battery: pd.DataFrame) -> pd.DataFrame:
135     complete = battery[battery["C1"].notna()].copy()
136     feature_columns = [
137         "C1", "Q1", "C2", "q", "T0", "E1", "E2", "J", "H",
138         "J_high_50", "J_high_60", "J_high_70", "structure_batch",
139     ]
140     response_columns = [
141         "soh200", "relative_loss200", "curve_loss200", "curve_linear", "curve_quadratic",
142         "mean_chargetime", "mean_IR", "mean_Tavg",
143     ]
144     aggregations: dict[str, str | tuple[str, str]] = {column: "first" for column in feature_columns}
145     aggregations.update({column: "mean" for column in response_columns})
146     aggregations["n_batteries"] = ("battery_id", "size")
147     result = complete.groupby("policy", as_index=False, observed=True).agg(**{
148         key: value if isinstance(value, tuple) else (key, value) for key, value in aggregations.items()
149     })
150     result["coordinate_id"] = coordinate_id(result)
151     result["equal_time_cohort"] = result["policy"].ne("3_6C-80PER_3_6C").astype(int)
152     result["explicit_new_structure_cohort"] = result["structure_batch"].eq(1).astype(int)
153     return result.sort_values("policy").reset_index(drop=True)
154
155
156 def standardized(train: pd.DataFrame, test: pd.DataFrame, features: tuple[str, ...]) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
157     if not features:
158         return np.empty((len(train), 0)), np.empty((len(test), 0)), np.array([], dtype=float), np.array([], dtype=float)
159     mean = train.loc[:, features].mean(axis=0).to_numpy(dtype=float, copy=True)
160     scale = train.loc[:, features].std(axis=0, ddof=0).to_numpy(dtype=float, copy=True)
161     scale[scale < 1e-12] = 1.0
162     return (
163         (train.loc[:, features].to_numpy(dtype=float) - mean) / scale,
164         (test.loc[:, features].to_numpy(dtype=float) - mean) / scale,
165         mean,
166         scale,
167     )
168
169
170 def ridge_fit(x: np.ndarray, y: np.ndarray, ridge_lambda: float) -> np.ndarray:
171     design = np.column_stack((np.ones(len(x)), x))
172     penalty = np.eye(design.shape[1]) * ridge_lambda
173     lhs = design.T @ design + penalty
174     return np.linalg.lstsq(lhs, design.T @ y, rcond=None)[0]
175
176
177
178 def ridge_predict(coef: np.ndarray, x: np.ndarray) -> np.ndarray:
179     return np.column_stack((np.ones(len(x)), x)) @ coef
180
181
182 def select_lambda_inner(train: pd.DataFrame, response: str, features: tuple[str, ...]) -> float:
183     groups = train["coordinate_id"].unique().tolist()
184     if len(groups) <= 3:
185         return 10.0
186     scores: list[tuple[float, float]] = []
187     for ridge_lambda in LAMBDA_GRID:
188         errors: list[float] = []
189         for group in groups:
190             inner_train = train[train["coordinate_id"].ne(group)]
191             inner_test = train[train["coordinate_id"].eq(group)]
192             x_train, x_test, _, _ = standardized(inner_train, inner_test, features)
193             coef = ridge_fit(x_train, inner_train[response].to_numpy(dtype=float), ridge_lambda)
194             prediction = ridge_predict(coef, x_test)
195             errors.append(float(np.mean(prediction - inner_test[response].to_numpy(dtype=float)) ** 2))
196         scores.append((float(np.mean(errors)), ridge_lambda))
197     minimum = min(value for value, _ in scores)

```

```

198     tolerance = minimum * 1.01 + 1e-15
199     return max(ridge_lambda for value, ridge_lambda in scores if value <= tolerance)
200
201
202 def nearest_prediction(train: pd.DataFrame, test: pd.DataFrame, response: str) -> np.ndarray:
203     features = ("C1", "q", "C2")
204     x_train, x_test, _, _ = standardized(train, test, features)
205     y = train[response].to_numpy(dtype=float)
206     output = np.empty(len(test), dtype=float)
207     for index, row in enumerate(x_test):
208         distance = np.sum((x_train - row) ** 2, axis=1)
209         output[index] = y[int(np.argmin(distance))]
210     return output
211
212
213 def fit_hierarchical_penalized(
214     train_cycles: pd.DataFrame,
215     train_strategy: pd.DataFrame,
216     features: tuple[str, ...],
217     quadratic: bool,
218     lambda_fixed: float = 1.0,
219     lambda_battery: float = 1.0,
220     lambda_policy: float = 10.0,
221 ) -> tuple[np.ndarray, dict[str, object]]:
222     """Fit a linearized smoke surrogate of Model C by penalized least squares.
223
224     y_it = beta0 + beta_x*x + sum beta_k*z_pk*x + beta_q*x^2
225           + a_i + u_i*x + v_p*x + error.
226
227     For runtime, this smoke fit uses cycle 1 and every fifth cycle. The formal
228     nonlinear exponential-link/AR(1) likelihood is intentionally not claimed
229     here. This surrogate answers whether the additional hierarchy has enough
230     predictive signal to justify a later full Model C fit.
231     """
232     strategy_lookup = train_strategy.set_index("policy")
233     frame = train_cycles[
234         train_cycles["policy"].isin(strategy_lookup.index)
235         & (train_cycles["cycle"].eq(1) | train_cycles["cycle"].mod(5).eq(0))
236     ].copy()
237     policies = sorted(frame["policy"].unique().tolist())
238     batteries = sorted(frame["battery_id"].unique().tolist())
239     p_index = {name: index for index, name in enumerate(policies)}
240     b_index = {value: index for index, value in enumerate(batteries)}
241     means = train_strategy.loc[:, features].mean(axis=0).to_numpy(dtype=float, copy=True) if features else np.array([])
242     scales = train_strategy.loc[:, features].std(axis=0, ddof=0).to_numpy(dtype=float, copy=True) if features else np.array([])
243     if len(scales):
244         scales[scales < 1e-12] = 1.0
245     x = frame["cycle"].to_numpy(dtype=float) / 200.0
246     fixed_columns = [np.ones(len(frame)), x]
247     for feature, mean, scale in zip(features, means, scales):
248         z = frame["policy"].map(strategy_lookup[feature]).to_numpy(dtype=float)
249         fixed_columns.append(((z - mean) / scale) * x)
250     if quadratic:
251         fixed_columns.append(x**2)
252     fixed = np.column_stack(fixed_columns)
253     random_battery = np.zeros((len(frame), 2 * len(batteries)))
254     random_policy = np.zeros((len(frame), len(policies)))
255     rows = np.arange(len(frame))
256     bidx = frame["battery_id"].map(b_index).to_numpy(dtype=int)
257     pidx = frame["policy"].map(p_index).to_numpy(dtype=int)
258     random_battery[rows, 2 * bidx] = 1.0
259     random_battery[rows, 2 * bidx + 1] = x
260     random_policy[rows, pidx] = x
261     design = np.column_stack((fixed, random_battery, random_policy))
262     penalty = np.zeros(design.shape[1])
263     penalty[2 : 2 + len(features)] = lambda_fixed
264     penalty[fixed.shape[1] : fixed.shape[1] + random_battery.shape[1]] = lambda_battery
265     penalty[fixed.shape[1] + random_battery.shape[1] :] = lambda_policy
266     lhs = design.T @ design
267     lhs.flat[:: lhs.shape[0] + 1] += penalty + 1e-10
268     rhs = design.T @ frame["SOH_clean"].to_numpy(dtype=float)
269     try:
270         coef = np.linalg.solve(lhs, rhs)
271     except np.linalg.LinAlgError:
272         coef = np.linalg.lstsq(lhs, rhs, rcond=None)[0]
273     fitted = design @ coef
274     residual = frame["SOH_clean"].to_numpy(dtype=float) - fitted
275     residual_frame = pd.DataFrame({"battery_id": frame["battery_id"].to_numpy(), "cycle": frame["cycle"].to_numpy(), "residual": residual})
276     pairs = []
277     for _, group in residual_frame.groupby("battery_id", observed=True):
278         values = group.sort_values("cycle")["residual"].to_numpy()
279         if len(values) > 1:
280             pairs.append(np.corrcoef(values[:-1], values[1:])[0, 1])
281     info = {
282         "fixed_coef": coef[: fixed.shape[1]],
283         "feature_mean": means,
284         "feature_scale": scales,
285         "features": features,
286         "quadratic": quadratic,
287         "train_rmse": float(np.sqrt(np.mean(residual**2))),
288         "lag1_residual_correlation": float(np.nanmean(pairs)),
289         "n_policy": len(policies),
290         "n_battery": len(batteries),
291     }
292     return coef, info
293
294
295 def predict_hierarchical(info: dict[str, object], test_strategy: pd.DataFrame, cycles: np.ndarray) -> np.ndarray:
296     features = tuple(info["features"])

```

```

297 means = np.asarray(info["feature_mean"], dtype=float)
298 scales = np.asarray(info["feature_scale"], dtype=float)
299 fixed_coef = np.asarray(info["fixed_coef"], dtype=float)
300 predictions: list[dict[str, float | int | str]] = []
301 x = cycles.astype(float) / 200.0
302 for _, row in test_strategy.iterrows():
303     columns = [np.ones(len(x)), x]
304     for feature, mean, scale in zip(features, means, scales):
305         columns.append(np.repeat((float(row[feature]) - mean) / scale, len(x)) * x)
306     if bool(info["quadratic"]):
307         columns.append(x**2)
308     value = np.column_stack(columns) @ fixed_coef
309     for cycle, prediction in zip(cycles, value):
310         predictions.append({"policy": row["policy"], "cycle": int(cycle), "prediction": float(prediction)})
311 return pd.DataFrame(predictions)

```

```

1 """Data loading, trajectory utilities, metrics, and constrained extrapolation."""
2
3 from __future__ import annotations
4
5 from dataclasses import dataclass
6 from pathlib import Path
7 from typing import Iterable
8
9 import numpy as np
10 import pandas as pd
11
12 from .config import CONFIG, Q3Config
13
14 @dataclass
15 class BatteryRecord:
16     battery_id: int
17     policy: str
18     meta: pd.Series
19     cycles: pd.DataFrame
20     baseline: float
21     relative_soh: np.ndarray
22
23     def relative_at(self, cycle: int) -> float:
24         return float(self.relative_soh[cycle - 1])
25
26     def absolute_future(self, start: int = 151, end: int = 200) -> np.ndarray:
27         return self.baseline * self.relative_soh[start - 1 : end]
28
29
30 def load_records(project_root: Path) -> tuple[dict[int, BatteryRecord], pd.DataFrame, pd.DataFrame]:
31     data_dir = project_root / "data" / "processed" / "q1_cleaned"
32     meta = pd.read_csv(data_dir / "battery_summary_clean.csv")
33     cycles = pd.read_csv(data_dir / "cycle_train_clean.csv")
34     records: dict[int, BatteryRecord] = {}
35     for battery_id, frame in cycles.groupby("battery_id", sort=True):
36         row = meta.loc[meta["battery_id"].eq(battery_id)].iloc[0]
37         frame = frame.sort_values("cycle").reset_index(drop=True)
38         baseline = float(frame.loc[frame["cycle"].between(1, 5), "SOH_clean"].mean())
39         records[int(battery_id)] = BatteryRecord(
40             battery_id=int(battery_id),
41             policy=str(row["policy"]),
42             meta=row,
43             cycles=frame,
44             baseline=baseline,
45             relative_soh=frame["SOH_clean"].to_numpy(float) / baseline,
46         )
47     return records, meta, cycles
48
49
50 def complete_battery_ids(meta: pd.DataFrame) -> list[int]:
51     return sorted(meta.loc[meta["prediction_test"].eq(0), "battery_id"].astype(int).tolist())
52
53
54 def slope(values: np.ndarray, cycles: np.ndarray | None = None) -> float:
55     values = np.asarray(values, dtype=float)
56     if cycles is None:
57         cycles = np.arange(1, len(values) + 1, dtype=float)
58     else:
59         cycles = np.asarray(cycles, dtype=float)
60     good = np.isfinite(values) & np.isfinite(cycles)
61     if good.sum() < 2:
62         return 0.0
63     x = cycles[good]
64     y = values[good]
65     xc = x - x.mean()
66     denom = float(xc @ xc)
67     return 0.0 if denom <= 0 else float(xc @ (y - y.mean()) / denom)
68
69
70 def robust_slope_scale(slopes: Iterable[float]) -> float:
71     values = np.asarray(list(slopes), dtype=float)
72     values = values[np.isfinite(values)]
73     if values.size == 0:
74         return 1e-8
75     median = float(np.median(values))
76     scale = 1.4826 * float(np.median(np.abs(values - median)))
77     if scale < 1e-8:
78         scale = max(float(np.std(values, ddof=1)) if values.size > 1 else 0.0, 1e-8)
79     return scale
80
81
82

```



```

83 def fit_power_law(
84     cycles: np.ndarray,
85     relative_soh: np.ndarray,
86     config: Q3Config = CONFIG,
87 ) -> dict[str, float]:
88     t = np.asarray(cycles, dtype=float)
89     y = np.asarray(relative_soh, dtype=float)
90     good = np.isfinite(t) & np.isfinite(y)
91     t, y = t[good], y[good]
92     if t.size < 5:
93         return {"beta0": float(np.nanmean(y)), "a": 0.0, "p": 1.0, "sse": np.inf}
94     L = float(t.max())
95     weights = 1.0 + 2.0 * (t / L) ** 2
96     root_w = np.sqrt(weights)
97     best: dict[str, float] | None = None
98     for p in config.power_grid:
99         z = t**p
100         design = np.column_stack([np.ones_like(t), -z])
101         coef, *_ = np.linalg.lstsq(design * root_w[:, None], y * root_w, rcond=None)
102         beta0, a = float(coef[0]), float(coef[1])
103         if a < 0:
104             a = 0.0
105             beta0 = float(np.average(y, weights=weights))
106             pred = beta0 - a * z
107             sse = float(np.sum(weights * (y - pred) ** 2))
108             candidate = {"beta0": beta0, "a": a, "p": float(p), "sse": sse}
109             if best is None or sse < best["sse"]:
110                 best = candidate
111     assert best is not None
112     return best
113
114
115 def predict_power_law(fit: dict[str, float], cycles: np.ndarray) -> np.ndarray:
116     t = np.asarray(cycles, dtype=float)
117     return fit["beta0"] - fit["a"] * t ** fit["p"]
118
119
120 def power_law_eol(fit: dict[str, float], baseline: float, config: Q3Config = CONFIG) -> tuple[float, str]:
121     threshold = 0.8 / baseline
122     beta0, a, p = fit["beta0"], fit["a"], fit["p"]
123     if not np.isfinite([beta0, a, p]).all() or a <= 0 or beta0 <= threshold or p <= 0:
124         return np.nan, "no_finite_intersection"
125     cycle = float(((beta0 - threshold) / a) ** (1.0 / p))
126     if not np.isfinite(cycle) or cycle > config.eol_max_cycle:
127         return np.nan, "beyond_5000"
128     if cycle <= 150:
129         return cycle, "before_or_at_observation"
130     return cycle, "finite_scenario"
131
132
133 def project_absolute_prediction(raw: np.ndarray, anchor: float, config: Q3Config = CONFIG) -> np.ndarray:
134     clipped = np.clip(np.asarray(raw, dtype=float), *config.soh_bounds)
135     return np.minimum.accumulate(np.concatenate([[float(anchor)], clipped]))[1:]
136
137
138 def prediction_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
139     residual = np.asarray(y_pred) - np.asarray(y_true)
140     return {
141         "rmse": float(np.sqrt(np.mean(residual**2))),
142         "mae": float(np.mean(np.abs(residual))),
143         "error_cycle200": float(residual[-1]),
144     }
145
146
147 def strategy_parameters(record: BatteryRecord) -> np.ndarray:
148     row = record.meta
149     return np.asarray([row.get("C1", np.nan), row.get("Q1", np.nan), row.get("C2", np.nan)], dtype=float)

```

```

1 """Auditable Q4 discrete Pareto and constrained single-exposure models."""
2
3 from __future__ import annotations
4
5 from dataclasses import dataclass
6 from pathlib import Path
7
8 import numpy as np
9 import pandas as pd
10
11 from q3_models.core import load_records
12
13
14 Q4_VERSION = "q4_smoke_v1"
15 SEED = 20260815
16 LAMBDA_GRID = np.array([0.0, 0.25, 0.5, 0.75, 1.0])
17
18
19 @dataclass(frozen=True)
20 class PolicyObservation:
21     policy: str
22     n_battery: int
23     time_mean: float
24     time_sd: float
25     loss_mean: float
26     loss_sd: float
27     late_slope_mean: float
28     c1: float
29     q1: float
30     c2: float

```

```

31 j: float
32 h: float
33 coordinate: tuple[float, float, float] | None
34
35
36 def _finite_mean(values: pd.Series) -> float:
37     values = pd.to_numeric(values, errors="coerce").dropna()
38     return float(values.mean()) if len(values) else float("nan")
39
40
41 def collect_policy_observations(project_root: Path) -> tuple[list[PolicyObservation], pd.DataFrame]:
42     records, meta, _ = load_records(project_root)
43     complete_ids = set(meta.loc[meta["prediction_test"].eq(0), "battery_id"].astype(int))
44     rows = []
45     observations: list[PolicyObservation] = []
46     for policy, group in meta.loc[meta["battery_id"].isin(complete_ids)].groupby("policy", sort=True):
47         battery_rows = []
48         for battery_id in sorted(group["battery_id"].astype(int)):
49             record = records[battery_id]
50             cycles = record.cycles
51             time_column = "chargetime_raw" if "chargetime_raw" in cycles.columns else "chargetime"
52             if time_column not in cycles.columns:
53                 raise KeyError("Q4 requires chargetime_raw (or legacy chargetime) in cleaned cycle data")
54             time_series = pd.to_numeric(cycles[time_column], errors="coerce")
55             time_value = float(time_series.mean())
56             loss_value = float(1.0 - record.relative_at(200))
57             late_values = record.relative_soh[150:200]
58             x = np.arange(151, 201, dtype=float)
59             xc = x - x.mean()
60             slope = float(xc @ (late_values - late_values.mean()) / (xc @ xc))
61             battery_rows.append({"battery_id": battery_id, "time": time_value,
62                                 "loss": loss_value, "late_slope_loss": max(0.0, -slope)})
63         values = pd.DataFrame(battery_rows)
64         first = group.iloc[0]
65         c1, q1, c2 = (float(first[col]) if pd.notna(first[col]) else np.nan for col in ("C1", "Q1", "C2"))
66         q = q1 / 100.0 if np.isfinite(q1) else np.nan
67         j = q * c1 + (0.8 - q) * c2 if np.isfinite([c1, q, c2]).all() else np.nan
68         h = 0.5 * (c1 * q**2 + c2 * (0.8**2 - q**2)) if np.isfinite([c1, q, c2]).all() else np.nan
69         coordinate = (c1, q1, c2) if np.isfinite([c1, q1, c2]).all() else None
70         observations.append(PolicyObservation(
71             policy=str(policy), n_battery=len(values), time_mean=float(values["time"].mean()),
72             time_sd=float(values["time"].std(ddof=1)) if len(values) > 1 else 0.0,
73             loss_mean=float(values["loss"].mean()),
74             loss_sd=float(values["loss"].std(ddof=1)) if len(values) > 1 else 0.0,
75             late_slope_mean=float(values["late_slope_loss"].mean()),
76             c1=c1, q1=q1, c2=c2, j=j, h=h, coordinate=coordinate,
77         ))
78         for row in battery_rows:
79             row.update({"policy": str(policy), "c1": c1, "q1": q1, "c2": c2, "j": j, "h": h})
80             rows.append(row)
81     return observations, pd.DataFrame(rows)
82
83
84 def observation_frame(observations: list[PolicyObservation]) -> pd.DataFrame:
85     return pd.DataFrame([obs.__dict__ for obs in observations])
86
87
88 def _normalise(values: np.ndarray) -> np.ndarray:
89     values = np.asarray(values, dtype=float)
90     lo, hi = np.nanmin(values), np.nanmax(values)
91     return np.zeros_like(values) if hi - lo < 1e-12 else (values - lo) / (hi - lo)
92
93
94 def pareto_mask(time: np.ndarray, loss: np.ndarray) -> np.ndarray:
95     time, loss = np.asarray(time, float), np.asarray(loss, float)
96     mask = np.ones(len(time), dtype=bool)
97     for i in range(len(time)):
98         dominated = (time <= time[i] + 1e-12) & (loss <= loss[i] + 1e-12)
99         strictly = (time < time[i] - 1e-12) | (loss < loss[i] - 1e-12)
100         mask[i] = not bool(np.any(dominated & strictly))
101     return mask
102
103
104 def choose_scalar(time: np.ndarray, loss: np.ndarray, policies: list[str], weight: float) -> int:
105     score = weight * _normalise(time) + (1.0 - weight) * _normalise(loss)
106     order = sorted(range(len(score)), key=lambda i: (score[i], loss[i], time[i], policies[i]))
107     return order[0]
108
109
110 def bootstrap_pareto(
111     frame: pd.DataFrame,
112     repetitions: int = 2000,
113     seed: int = SEED,
114     lambda_grid: np.ndarray | None = None,
115     loss_limits: tuple[float, ...] = (),
116 ) -> pd.DataFrame:
117     rng = np.random.default_rng(seed)
118     lambda_grid = LAMBDA_GRID if lambda_grid is None else np.asarray(lambda_grid, dtype=float)
119     policies = sorted(frame["policy"].unique())
120     records = {policy: frame.loc[frame["policy"].eq(policy)].reset_index(drop=True) for policy in policies}
121     rows = []
122     for rep in range(repetitions):
123         summary = []
124         for policy in policies:
125             data = records[policy]
126             idx = rng.integers(0, len(data), size=len(data))
127             sample = data.iloc[idx]
128             late_slope = float(sample["late_slope_loss"].mean()) if "late_slope_loss" in sample else np.nan
129             summary.append((policy, float(sample["time"].mean()), float(sample["loss"].mean()), late_slope))

```

```

130 t = np.array([row[1] for row in summary]); d = np.array([row[2] for row in summary])
131 front = pareto_mask(t, d)
132 choices = {w: summary[choose_scalar(t, d, policies, w)][0] for w in lambda_grid}
133 constrained_choices = {}
134 for limit in loss_limits:
135     feasible = np.flatnonzero(d <= limit)
136     constrained_choices[limit] = (
137         min(feasible, key=lambda i: (t[i], d[i], policies[i])) if len(feasible) else None
138     )
139 for index, ((policy, time_value, loss_value, late_slope), is_front) in enumerate(zip(summary, front)):
140     rows.append({"version": Q4_VERSION, "replicate": rep, "policy": policy,
141                 "time": time_value, "loss": loss_value, "late_slope_loss": late_slope,
142                 "pareto": bool(is_front),
143                 **{f"selected_lambda_{w:.2f}": choices[w] == policy for w in lambda_grid},
144                 **{f"selected_loss_limit_{limit:.4f}": constrained_choices[limit] == index
145                    for limit in loss_limits}})
146 return pd.DataFrame(rows)
147
148
149 def fit_single_exposure(train: pd.DataFrame, exposure: str, lam: float) -> tuple[float, float, float, float]:
150     loss_column = "loss" if "loss" in train.columns else "loss_mean"
151     train = train.dropna(subset=[exposure, loss_column])
152     if len(train) < 2 or train[exposure].std(ddof=0) < 1e-8:
153         return float(train[loss_column].mean()), 0.0, float(train[exposure].mean()), 1.0
154     mean, scale = float(train[exposure].mean()), float(train[exposure].std(ddof=0))
155     z = (train[exposure].to_numpy(float) - mean) / scale
156     x = np.column_stack([np.ones(len(z)), z]); y = train[loss_column].to_numpy(float)
157     penalty = np.diag([0.0, lam]); coef = np.linalg.solve(x.T @ x + penalty, x.T @ y)
158     return float(coef[0]), float(coef[1]), mean, scale
159
160
161 def loso_single_exposure(
162     frame: pd.DataFrame,
163     exposure: str = "j",
164     ridge_grid: tuple[float, ...] = (0.01, 0.1, 1.0, 10.0),
165 ) -> pd.DataFrame:
166     usable = frame.dropna(subset=[exposure, "coordinate", "loss_mean"]).copy()
167     usable["coordinate_key"] = usable["coordinate"].astype(str)
168     rows = []
169     for held_out in sorted(usable["coordinate_key"].unique()):
170         train = usable.loc[usable["coordinate_key"] != held_out]
171         test = usable.loc[usable["coordinate_key"] == held_out]
172         best = None
173         for lam in ridge_grid:
174             intercept, slope, mean, scale = fit_single_exposure(train, exposure, lam)
175             if len(test) == 0 or scale < 1e-8:
176                 pred = np.full(len(test), intercept)
177             else:
178                 pred = intercept + slope * (test[exposure].to_numpy(float) - mean) / scale
179             rmse = float(np.sqrt(np.mean((pred - test["loss_mean"]).to_numpy(float)) ** 2)) if len(test) else np.nan
180             candidate = (rmse, -lam, intercept, slope, mean, scale)
181             if best is None or candidate[2] < best[2]: best = candidate
182         assert best is not None
183         baseline_rmse = float(np.sqrt(np.mean((test["loss_mean"]).to_numpy(float) - train["loss_mean"].mean()) ** 2)) if len(test) else np.
184         nan
185         rows.append({"version": Q4_VERSION, "exposure": exposure, "held_out_coordinate": held_out,
186                     "n_test_policy": len(test), "rmse": best[0], "lambda": -best[1],
187                     "constant_rmse": baseline_rmse, "improvement": baseline_rmse - best[0],
188                     "intercept": best[2], "slope": best[3], "train_mean": best[4],
189                     "train_scale": best[5], "validation_type": "coordinate_LOSO_extrapolation_pressure"})
189     return pd.DataFrame(rows)

```

附录 C AI 工具使用详情

本参赛队在竞赛中使用 AI 工具辅助完成以下环节：

1. 代码调试与运行环境配置；
2. 语言润色；
3. 数据处理与建模方案讨论。

核心建模思路、模型选择、数值结果核验与最终结论均由参赛队独立完成，AI 生成的代码与文字均已逐项人工审查与核实。详细交互过程见支撑材料”AI 工具使用详情.pdf”。